



Image Optimization Tool

Installation & Setup Guide

Version 2.0 | Production-Ready with Security Enhancements

Updated: January 2026



Table of Contents

1. Overview
2. System Requirements
3. Installation Steps
4. Installation Verification
5. Quick Start Guide
6. Usage Examples
7. Troubleshooting
8. Uninstallation

1. Overview

The Image Optimization Tool (`imgtool`) is a production-ready CLI application for batch image processing with enterprise-grade security features.

Key Features

- Format Support** HEIC, PNG, WebP, TIFF → optimized JPEG
- Security** Path traversal protection & resource limits
- Quality** Smart JPEG handling prevents generation loss

- **Organization** Automatic size-based bucketing
- **Audit Trail** CSV/JSON reports for compliance

2. System Requirements

Minimum Requirements

Component	Requirement
Operating System	Linux, macOS, Windows (with Python support)
Python Version	Python 3.8 or higher
Disk Space	100 MB for dependencies + space for output images
RAM	2 GB minimum (4 GB recommended for large images)

Software Prerequisites

- **Python 3.8+** - Download from python.org
- **pip** - Python package installer (included with Python 3.4+)
- **Git** (optional) - For cloning the repository

3. Installation Steps

Step 1: Verify Python Installation

Open a terminal/command prompt and run:

```
# Check Python version
python3 --version

# Expected output: Python 3.8.0 or higher
```

⚠ **Windows Users:** Use `python` instead of `python3` on Windows.
⚠ **macOS Users:** Ensure you're using Python 3, not the system Python 2.7.

Step 2: Download the Tool

Option A: Clone via Git (Recommended)

```
# Clone the repository
git clone https://github.com/emack001/jmrtradingllc.github.io.git

# Navigate to scripts directory
cd jmrtradingllc.github.io/scripts
```

Option B: Direct Download

1. Download the repository as ZIP from GitHub
2. Extract the archive
3. Navigate to the `scripts/` folder

Step 3: Install Dependencies

Required Dependencies

```
# Navigate to scripts directory
cd scripts/

# Install required packages
pip install -r requirements.txt

# Or install Pillow directly
pip install "Pillow>=10.0.0"
```

Optional Dependencies

For HEIC/HEIF Support (iPhone/Mac photos):

```
pip install pillow-heif
```

For Advanced EXIF Manipulation:

```
pip install piexif
```



Complete Installation: To install all dependencies including optional ones:

```
pip install Pillow pillow-heif piexif
```

Step 4: Make Script Executable (Linux/macOS)

```
# Make the script executable
chmod +x imgtool.py

# Verify permissions
ls -l imgtool.py
```

4. Installation Verification

Test 1: Display Help Message

```
# Run the help command
python3 imgtool.py --help
```

Expected output should show usage information and available options.

Test 2: Check Dependency Support

```
# Test with verbose output
python3 imgtool.py /nonexistent --dry-run -v 2>&1 | grep -i "support"
```

You should see messages about HEIC/HEIF support status.

Test 3: Dry-Run Test

Create a test directory with sample images and run:

```
# Create test directory
mkdir -p ~/test-images

# Run dry-run (preview mode)
python3 imgtool.py ~/test-images --dry-run -v
```

✓ **Installation Successful!** If all tests pass, the tool is ready to use.

5. Quick Start Guide

Basic Workflow

1. **Prepare your images** - Place images in a folder (e.g., `~/Photos`)
2. **Run the tool** - Execute with desired options
3. **Check output** - Review organized images in buckets
4. **Review reports** - Check CSV/JSON for audit trail

Your First Conversion

```
# Convert all images in a folder
python3 imgtool.py ~/Photos

# Output will be in:
# ./cleaned/small/      (≤ 200 KB)
# ./cleaned/medium/    (≤ 600 KB)
# ./cleaned/large/     (≤ 1500 KB)
# ./cleaned/xlarge/    (> 1500 KB)
```

6. Usage Examples

Example 1: iPhone/Mac Photos (HEIC)

```
python3 imgtool.py ~/iPhone-Photos \  
-r \  
--fix-orientation \  
--preserve-exif \  
-o ~/Organized-Photos
```

What this does:

- `-r` - Recursively process subfolders
- `--fix-orientation` - Correct rotated images
- `--preserve-exif` - Keep camera metadata
- `-o` - Custom output directory

Example 2: Web Asset Optimization

```
python3 imgtool.py ./website-images \  
-r \  
--max-dim 1920 \  
-q 90 \  
--keep-structure
```

What this does:

- `--max-dim 1920` - Resize to max 1920px (Full HD)
- `-q 90` - High quality JPEG (90%)
- `--keep-structure` - Mirror folder structure

Example 3: Safe Preview (Dry-Run)

```
python3 imgtool.py ~/important-photos --dry-run -vv
```

What this does:

- `--dry-run` - Preview without making changes
- `-vv` - Maximum verbosity (debug mode)

Example 4: Production Batch with Security Limits

```
python3 imgtool.py /data/uploads \  
-r \  
--max-filesize-mb 50 \  
--max-pixels 25000000 \  
-q 85 \  
--log-file /var/log/imgtool.log \  
-v
```

What this does:

- `--max-filesize-mb 50` - Skip files larger than 50MB
- `--max-pixels 25000000` - Skip images over $\sim 5000 \times 5000$
- `--log-file` - Save logs to file for auditing

7. Troubleshooting

Issue: "No module named 'PIL'"

Solution:

```
pip install Pillow
```

Ensure you're using the correct pip for your Python version.

Issue: "HEIC/HEIF support missing"

Solution:

```
pip install pillow-heif
```

On some systems, you may also need to install system libraries:

Ubuntu/Debian: `apt-get install libheif-dev`

macOS: `brew install libheif`

Issue: "Permission denied" when running script

Solution (Linux/macOS):

```
chmod +x imgtool.py  
./imgtool.py --help
```

Solution (Windows): Always use `python imgtool.py` instead of running directly.

Issue: Out of Memory

Solution: Use resource limits to process smaller batches:

```
python3 imgtool.py ~/images \  
--max-filesize-mb 50 \  
--max-pixels 25000000
```

Issue: "Input must be a directory"

Solution: Ensure you provide a folder path, not a file:


```
# ❌ Wrong
python3 imgtool.py photo.jpg

# ✅ Correct
python3 imgtool.py ~/Photos
```

8. Uninstallation

Remove Python Dependencies

```
# Uninstall packages
pip uninstall Pillow pillow-heif piexif -y
```

Remove Script Files

```
# Navigate to parent directory
cd ..

# Remove the scripts directory
rm -rf scripts/
```

Remove Output Files (Optional)

```
# Remove default output directory
rm -rf cleaned/

# Remove custom output directories
rm -rf ~/Organized-Photos/
```



Support & Resources

Resource	Location
Documentation	<code>scripts/README.md</code>
Technical Details	<code>scripts/ENHANCEMENTS.md</code>
Dependencies	<code>scripts/requirements.txt</code>
Source Code	<code>scripts/imgtool.py</code>
GitHub Repository	emack001/jmrtradingllc.github.io



Security Features (v2.0)

Enhanced Security Protections

- **Path Traversal Protection** - Prevents malicious filenames from escaping output directory
- **Resource Limits** - Configurable max file size (200MB) and resolution (89MP)
- **Quality Preservation** - Skips JPEG re-compression by default to prevent generation loss

These features are enabled by default and require no configuration.



Post-Installation Checklist

- ☐ Python 3.8+ installed and verified
- ☐ Pillow package installed successfully
- ☐ Optional dependencies installed (if needed)

- ☐ Script runs with `--help` flag
- ☐ Dry-run test completed successfully
- ☐ Documentation reviewed
- ☐ Test images processed successfully

 **You're all set!** The Image Optimization Tool is ready to process your images with enterprise-grade security and quality preservation.

Image Optimization Tool v2.0 | Installation Guide | January 2026

© 2026 JMR Trading LLC | Licensed under MIT